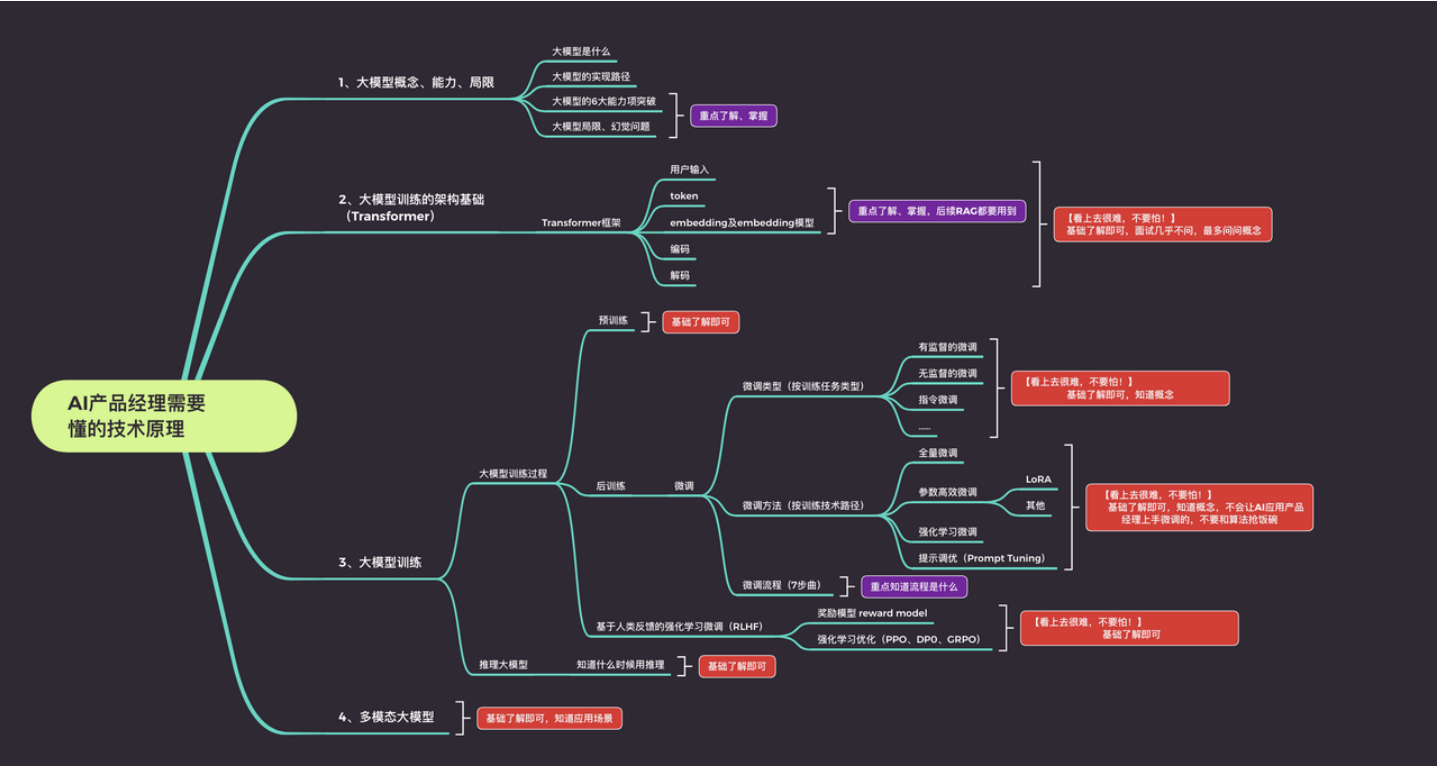


02-大模型关键技术知识（补充知识）

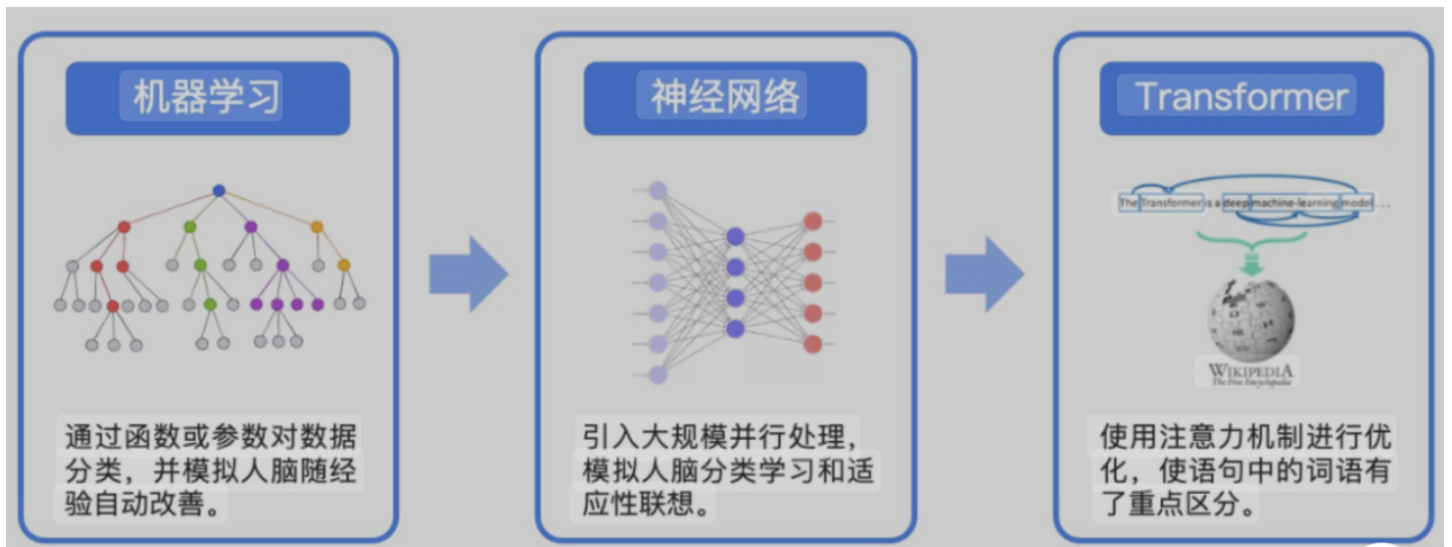
0、本节课学习地图



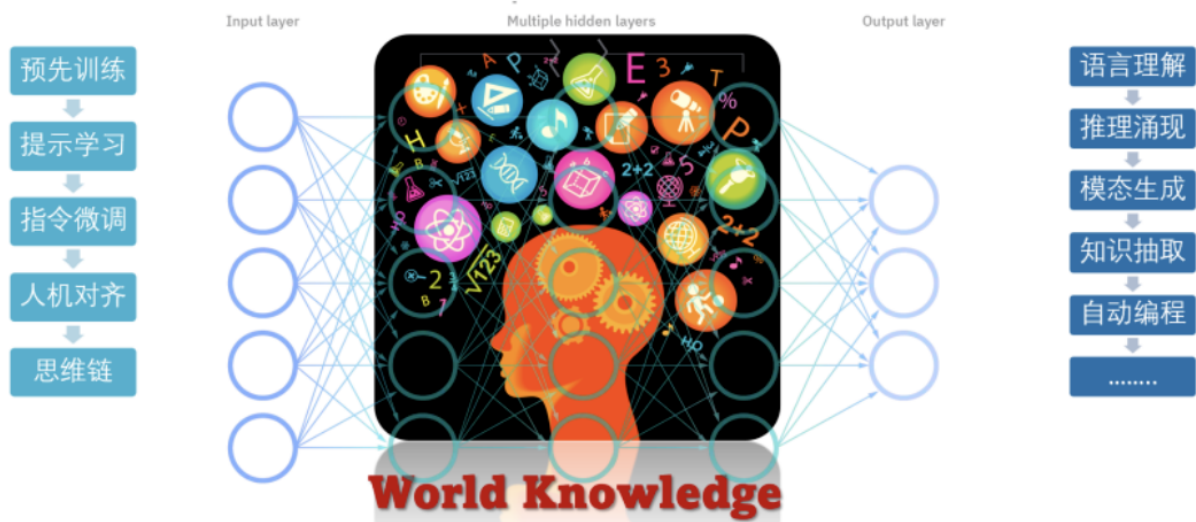
1、大模型概念、能力、局限

1.1 深度神经网络

人工智能的底层模型是“神经网络”（neural network），大模型是由“深度神经网络”构建出的，但远不止于“深”——它是“更大、更广、更能泛化的深度神经网络”。



世界知识的神经网络化表示 + 超强的AI生成能力



ChatGPT是**知识模型**与**算法模型**的混合体：大模型将海量知识（以文字序列为主）预先学习进神经网络中，与传统以算法为主的模型不同，**ChatGPT**训练的是**富含知识的模型**

代码块

- 1 **深度神经网络就像“发动机”**：图像识别、语音识别、推荐系统背后都靠它。
- 2
- 3 **大模型就像“整辆高端自动驾驶汽车”**：有强大的“语言引擎”、巨大的“知识地图”、超强“适配能力”，能一车多用，适应各种任务。

(1) 大模型 vs 神经网络 对比：

项目	深度神经网络（DNN）	大模型（Large Model / LLM）
定义	多层神经网络组成的模型，用于特征提取与学习	参数数量极大（亿~千亿）、支持多任务/多语言的大规模模型
架构	包括 CNN、RNN、Transformer 等	一般基于 Transformer（如 GPT、T5、BERT）
参数量	通常几十万到几千万参数	动辄上亿、百亿甚至千亿参数（如GPT-4、PaLM 2）
任务范围	单一任务（如图像分类、情感分析）	多任务通用模型（可问答、写文案、编程、摘要等）
训练数据	结构化 / 小规模	万亿token级别的大语料，多语言、多模态数据
是否可微调	通常专门训练	支持提示工程（Prompting）、LoRA、SFT等多种调优方式
是否理解“语言”	依任务而定，需特定设计	内建语言模型能力，能理解自然语言提示
举例	图像识别、情感分析	GPT-3/4、Claude、通义千问、文心一言等

1.2 大模型底层实现步骤（3步）

预训练像“读万卷书”，微调像“习题训练”，RLHF 则是“老师讲评 + 修正风格”

（1）Step 1：预训练（Pre-training）

举例：像是让一个孩子读遍全世界的书，看 Wikipedia、新闻、小说、论坛、说明书，虽然还没上学考试，但已经积累了大量常识和语言能力。

代码块

- 1 广泛阅读 → 学会语言结构 + 基础常识
- 2 数据量大、知识广、任务无明确指令

（2）Step 2：有监督微调（Supervised Fine-Tuning）

举例：孩子上学了，老师教孩子如何写作文，给小孩看优秀作文300篇，让小朋友学会写高分作文。

代码块

- 1 明确指令 + 正确示范
- 2 数据量中等，针对性强，模型开始“会听懂人的指令”

（3）Step 3：人类反馈强化学习（RLHF）

举例：孩子交作文，老师打分：这里写的不错很流畅，那里写的太啰嗦，中心思想不明确

孩子就去根据老师的反馈修改作文内容

代码块

1

模型学会了一个新的能力：不仅是“答对”，而是“让人喜欢”、“更像人类”

2

3

用人类反馈“奖励” → 优化模型行为

4

昂贵但非常重要，可以提升模型能力 & 减少幻觉

1.3 大模型3大局限

自进化和个性化问题

大模型很难区分“知识、偏差、个性”，同时定制成本高，难以做到低成本、实时、个性适配。

(1) 模型无法区分：新知识 vs 错误反馈 vs 个性偏好

类别	示例	问题点
新知识	“GPT请问OpenAI最新发布了什么？”	模型必须持续学习新数据
错误反馈	用户说“你错了”，其实模型没错，是用户搞错了	模型容易误学、变差
个性偏好	“我喜欢用表格总结回答”	模型需要“只对你变聪明”，不能影响其他人

大模型天生缺乏“个体记忆”能力

(2) 个性化的难度：成本高 + 不易维护 + 易干扰主模型

代码块

1

大模型参数量巨大，每次个性微调成本高

2

个性化与泛化冲突：改过之后对A用户变好，对B用户有可能变差

3

难以持续更新

4

存储&调度复杂：每个用户一套模型/权重/记忆？ → 资源抗不住，会爆炸

(3) 典型失败的类型：

风险类型	说明
模型学错	用户恶意/误反馈被当作“真实知识”学习了
泛化能力下降	某个用户习惯被过度强化，输出变得非通用
安全风险	个性化学习中误采纳有害内容（如仇恨、偏见）
成本不可控	成千上万个用户微调 → 存储、调用、部署开销大

(4) 如何解决，靠外力！

代码块

1

在 Prompt 中加入用户偏好

2

RAG 外挂知识库解决知识更新问题

3

Agent：配上手脚，执行个性化工作及任务

多模态大模型训练有效方式仍有待探索

多模态大模型 = 能够同时处理文本、图像、语音、视频等多种数据模态的大模型（如Gemini、Claude 3等）

(1) 为什么难？

代码块

1

1、不同模态语义空间不统一：

2

图像是二维像素矩阵，文本是线性离散Token，模型要“对齐”这些模态的表达方式（叫**跨模态对齐**），非常困难

3

4

2、缺乏大规模高质量对齐数据

5

文本数据丰富，图文、音图、视频+文字数据却非常稀缺（尤其是标注好的）

6

7

3、训练方式复杂且缺乏统一的评估与任务标准

(2) 当前阶段：

文本大模型已进入应用落地期，多模态大模型仍处在“基础能力预训练 + 模态对齐”阶段，还没跑通稳定的商业训练范式。

2、大模型训练的架构基础（Transformer）

2.1 自注意力机制（Self-Attention）——了解即可

让每个词在理解句子时，能够“关注到句子中和自己有关的所有其他词”，并据此决定自己该怎么理解。

自注意力机制让语言模型不再“逐词死记”，而是“全局对话理解”。

再举个例子：

“她把苹果给了小明，因为他饿了。”

请问：“他”是谁？

代码块

- 1 人类可以理解“他”是“小明”
- 2 模型也要学会理解“他”和“小明”之间有依赖关系
- 3 这时，自注意机制会让“他”去“关注”整句话中与自己相关的词—发现“小明”和“饿”最相关
- 4 输出语义中，就包含了“饿的小明”的信息

这就是 Self-Attention 的核心作用：自动捕捉上下文中重要关联

2.2 Transformer 框架——了解即可

Transformer 是一种让模型“看清整体上下文、并平行处理信息”的神经网络结构，它的核心在于“自注意力机制”，让每个词都能关注到和自己相关的所有词。

1、为什么要有 Transformer？

代码块

- 1 因为传统模型有2个最大的问题：
- 2
- 3 (1) 只能一个词一个词读，不能并行，速度慢
- 4
- 5 (2) 长句子越往后，越记不住前面说了啥（长依赖问题）

Transformer 可以一次性“扫一眼全文”，让模型不仅能“知道下一个词”，每个词都能理解整个句子的关系，而且能同时处理很多词

2、Transformer 包括：Encoder编码器 和 Decoder解码器两部分：

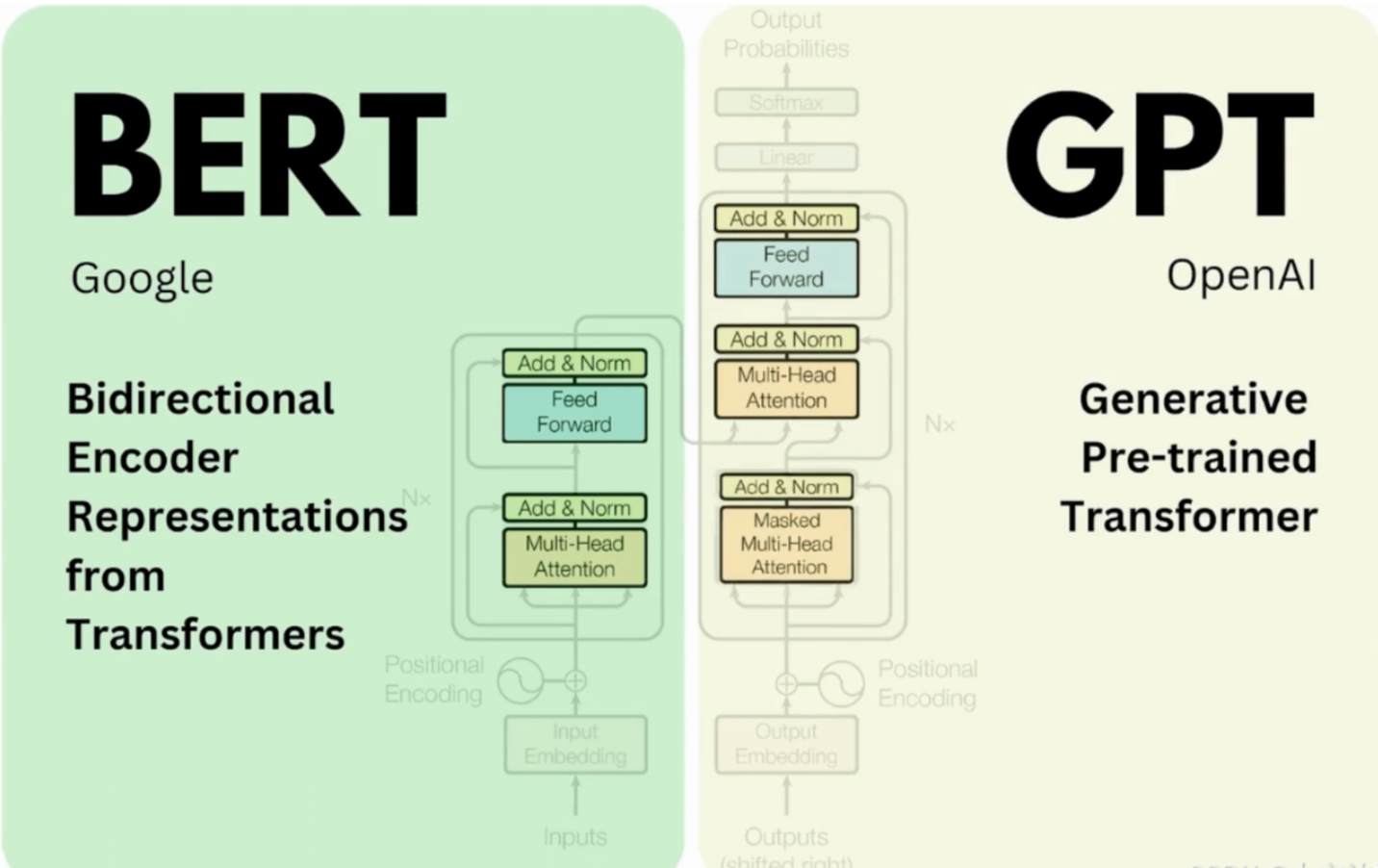
模块	用途	举例
Encoder	把一句话“理解”成语义向量	文本分类、情感分析
Decoder	根据上下文逐字“生成”内容	自动写诗、翻译等

2.3 最典型的两种预训练大模型：BERT模型、GPT模型

google2018年 首次推出BERT大语言模型，OpenAI2018年首次推出GPT大语言模型，这两种都是基于Transformer架构的预训练模型，Transformer 是它们的“祖师爷”

代码块

- 1
- Transformer像是一个通用的“语言大脑结构图”，
- 2
-
- 3
- BERT 是用它来学“怎么理解话”—专注理解
- 4
- GPT 是用它来学“怎么说话”—专注生成



1、Bert vs GPT核心区别：使用 Transformer 的哪一半

模型组件	Transformer 原始结构	BERT 用了什么	GPT 用了什么
Encoder 编码器	✅（用于理解输入，更适合分类、抽取）	✅ 只用 Encoder	❌ 不使用
Decoder 解码器	✅（用于生成输出），更适合续写、对话	❌ 不使用	✅ 只用 Decoder

举例：中译英翻译任务

代码块

1

输入：“我今天心情很好。”

2

3

BERT：可以判断这是一个积极情绪的句子（分类）

4

GPT：可以续写这个句子：“我今天心情很好，所以我.....”

2、有没有同时用了encoder 和 decoder的大模型？

模型名称	开源机构	特点
T5	Google（谷歌）	开创了将所有任务都转化为“文本 → 文本”的格式处理
FLAN-T5	Google（谷歌）	在 T5 基础上加入 instruction tuning 指令微调
UL2	Google（谷歌）	更通用的 Encoder-Decoder 模型，兼容 T5，支持多种预训练目标

代码块

1

兼顾理解 + 生成，适合多任务处理

2

Encoder：负责读取输入（如问题、文章、翻译原文），转化成token, embedding

3

Decoder：基于生成输出文本（如回答、摘要、译文）

3、chatgpt 没有encoder这个环节，但是不是也能理解人类意图么，怎么做到的？

虽然 ChatGPT 只用了 Transformer 的 Decoder 架构，但它通过“语言生成”的方式，也可以完成分类、理解等任务——因为分类本质上可以被“转化成生成”来实现。

举例：意图分类：


```
1 用户输入：“我想看看明天的天气”
2
3 (1) 分类模型bert输出：[查天气]
4
5 (2) chatgpt的处理方式是：
6 Prompt：“请判断这句话的意图类别。”
7 模型生成输出：“这句话的意图是：查询天气”
8 虽然它是在“生成答案”，但这就是一种分类
```

所以，一切任务都可以“转成生成”

2.4 上下文长度（Context Length）

大模型在一次输入中能“看见”多少个 token（词片段），即模型的“短期记忆容量”。

代码块

```
1 “大模型一次性能读多少内容、记住多少信息、再基于这些内容给你回应”。
```

（1）到底多长？

代码块

```
1 if模型的【上下文长度】是 64k tokens（约5w中文），只要总长不超过 64k，它就能“理解这些上
  下文”，输出基于全局的信息。
2
3 在中文环境下，1 个 token ≈ 0.75 个汉字
4 64,000 tokens × 0.75 ≈ 48,000 汉字 ≈ 5 万中文字符
```

应用场景	实际能力
阅读长文档	一次性输入一整份1~2万字的报告/PDF，无需拆分
多轮对话记忆	保留用户历史几十轮对话，理解上下文一致性
高质量RAG	可以从知识库中引入更多召回文段，提升回答准确性
多段代码处理	输入完整项目代码 + 报错信息 + 用户提问，适用于AI编程助手
AI办公场景	可以处理整份PPT内容、长邮件线程、汇报材料等

(2) 超出上下文64k token怎么办？

代码块

- 1 会被截断，模型只读取 64k token
- 2 截断方式默认是“从前往后截”或“从后往前截”（依模型而定）

(3) 对产品设计的影响：

需要考虑的问题	建议
提示构造（Prompt）	控制 Token 数，防止“超限截断导致信息缺失”
对话类任务	对历史内容做摘要、合并、记忆提取后再塞进 Prompt（以免多轮对话太长）
多文档问答	加入内容筛选 / 多轮召回 等
多模态输入	如果有图 + 文混合，注意图像转描述后Token数暴涨
成本控制	上下文越长，推理成本越高，不要“无脑塞文档”

2.5 向量

数据转换成向量的过程称为 embedding 向量化，是通过embedding模型实现的

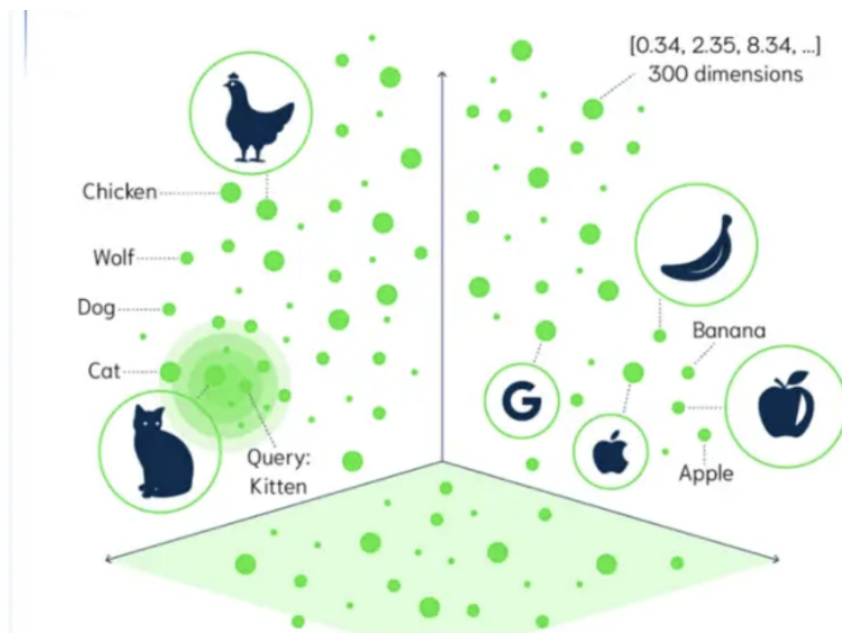
代码块

- 1 Embedding 向量是将“语言、图片、物体、用户”等原始信息，转化为计算机能理解的数字向量表示，使模型能够“计算、比较、理解”它们。

哲学三问：

- ? 向量是什么
- ? 向量从哪来
- ? 向量用到哪去

- 向量是在多维数学空间的坐标，是一个点
- 向量来源于对真实世界数字化的抽象
- 向量用来搜索真实世界的数据库



举例：人类理解“苹果”和“香蕉”是通过感觉、经验；而计算机不懂这些词，它必须把每个词转换成数字向量，比如：

代码块

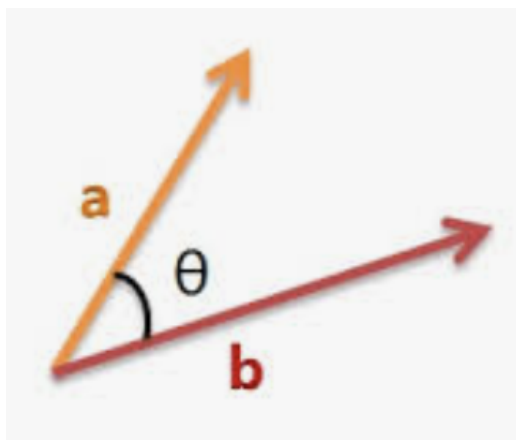
- “苹果” $\rightarrow [0.12, -1.3, 0.75, \dots]$
- “香蕉” $\rightarrow [0.10, -1.2, 0.72, \dots]$

这就叫做**词嵌入（word embedding）**，或者**语义向量表示**。

Embedding关键特性：

特性	举例
语义相近 $\rightarrow$ 向量也相近	“猫”和“狗”的向量距离小；“猫”和“飞机”的距离大
可以用于相似度计算	用 余弦相似度 判断两个文本、商品、用户是否相似
统一空间表示不同对象	不只是文本，图像、商品、用户也能被Embedding表示

余弦相似度：两个向量的夹角，余弦值越接近1，就表明夹角越接近0度，也就是两个向量越相似



3、大模型训练

3.1 ChatGPT 训练4步曲

阶段	名称	全称	作用
1	预训练	Pretraining	让模型学会“语言本能”
2	指令微调	Supervised Fine-Tuning (SFT)	教模型听懂人类指令
3	奖励模型训练	Reward Model (RM)	建立“好回答 vs 坏回答”的评判标准
4	强化学习微调	Reinforcement Learning with Human Feedback (PPO)	用人类偏好优化生成策略

- ✓

预训练给了大模型：“知识”

指令微调给了大模型：“任务能力”

奖励模型给了大模型：“判断力”

PPO优化给了大模型：“人类偏好”

1、预训练（Pretraining）：搭建“模型大脑”，但他还不懂你要干啥

代码块

1

输入：互联网上的海量文本（网页、维基百科、书籍、代码等）

2

3

任务：预测下一个词（语言建模）

4

5

如：

6

输入：“我今天心情特别” → 预测下一个词是“好”还是“糟糕”

7

结果：模型掌握“语言能力 + 世界常识 + 模式规律”

2、指令微调（SFT）：从“语言机器人”到“指令助手”

代码块

1

输入：人类写的“问题-好答案”对（高质量指令数据）

2

3

比如：

4

用户：请写一首五言绝句

- 5 模型答案：日落江湖白，潮来天地青。...
- 6
- 7 模型学习：遇到类似任务 → 模仿答案风格 → 学会“听人类话”

3、奖励模型（Reward Model）：“教TA怎么打分”：建立“好/坏回答”的打分系统，为后续**强化学习做准备**：怎么奖励、怎么惩罚

代码块

- 1 方式：给模型两种回答 → 让人类打分 → 哪个更好？
- 2
- 3 奖励模型学习：“好回答”的语气、逻辑、礼貌、精炼程度
- 4 输出：一个模型可以对任意输出“打分”，形成奖励函数

4、PPO优化（强化学习 with 人类反馈）：**用打分系统来“反向优化”模型的输出策略** → 让模型生成更符合人类偏好的答案。

代码块

- 1 Proximal Policy Optimization (PPO) 是一种算法
- 2
- 3 步骤：
- 4 模型生成多个回答 → 奖励模型给分 → PPO算法优化策略
- 5
- 6 结果：
- 7 输出更符合人类喜好
- 8 回答更安全、更稳重、不容易幻觉、不容易攻击性内容

3.2 大模型的参数

“7B”、“60B” 是大模型的参数规模指标，**在深度神经网络中，“参数”是模型用来学习和记住知识的东西，对模型能力、成本、速度都有重要影响**

代码块

- 1 **7B = 7 Billion = 70亿个参数**
- 2 **60B = 60 Billion = 600亿个参数**

(1) 参数大小的差异：

类比	参数少（如7B）	参数多（如60B）
模型像	高中生	博士生
记忆力	看过几本书	看过几千本书
表达能力	能写报告	能写小说、辩论
成本	一台电脑跑得动	需要云服务器支持

(2) 参数大小影响什么？

项目	小模型（<13B）	大模型（60B+）
知识记忆量	少（简单任务）	多（常识、推理、创作）
推理能力	弱	强（可支持多步逻辑）
语言生成质量	简单句子、模板化	句子流畅、自然、有风格
多任务泛化	容易过拟合	泛化能力强
部署成本	低，手机也能跑	高，需GPU或云端多卡
推理速度	快	慢（资源吃紧）
使用成本	便宜	昂贵（模型越大越贵）

(3) 参数多 意味着一定更好？

代码块

- 1
- (1) 如果训练数据不好、指令微调不到位，大模型也可能答得差
- 2
- (2) “小模型+好微调+好Prompt”效果比“大模型裸用”还好，要结合实际业务场景来看
- 3
- (3) 越大的模型，推理速度越慢、成本越高、不适合移动端部署

3.3 大模型生成为什么“每次都不一样”

大模型生成不是“永远选最可能的词”，而是按概率进行采样，这才能让它更自然、更有创造力、更像人类说话。

(1) 技术原理：模型输出的是“概率分布”

代码块

- 1
- if 你给出一句话：“花间一壶酒，独酌无相亲。举杯邀明月，对影成”
- 2
- 模型预测下一个字，会输出类似这样的概率分布：

词	概率
“三人”	70% 
“双影”	20%
“孤影”	9%
“烤鱼”	1% 

如果 总选最高概率的“70%”那个，就永远是“成三人” → 每次都一样 → 很无聊，这和人类是不一样的，因为人

有时会灵感一现、换一种表达方式、犯点小错误。

(2) 为什么不让大模型“永远选最可能的词”？

只选最高概率（贪婪）	按概率采样
句子更稳定	更有变化
风格单一，容易重复	更像人写的、有惊喜
容易卡住（死循环）	可以突破句型、写诗、续写故事
不适合创作任务	非常适合文案、对话、写作等场景（回归到生成，生成就该是有变化的）

(3) 控制“随机性”的参数：Temperature（温度）

Temperature 值	效果
0	极度保守 → 永远选最大概率词（几乎固定输出）
0.7	默认推荐 → 稍有随机性，风格自然稳定
1.0+	高随机性 → 输出更丰富，但可能错乱
>1.5	基本疯了 → 易出幻觉、胡说八道

以扣子平台 LLM配置为例：



3.4 指令微调

将预训练完成的大模型，用人工构造的“指令-回答”数据集进行再训练，让它学会“执行任务、听懂人话”。

(1) 流程：

代码块

- 1 【预训练模型】如deepseek V3
- 2 ↓
- 3 【收集“任务指令 → 回答”数据】
- 4 ↓
- 5 【统一 Prompt 格式】
- 6 ↓
- 7 【LoRA/全参微调训练】
- 8 ↓
- 9 【得到“能听人话、会干活”的语言模型】

(2) 示例：

a)准备高质量 “指令-输出” 对

内容	示例
指令	如“请帮我写一段旅游文案，目的地是厦门”
输出	“厦门，海风拂面，文艺又热情...”
数据来源	人工写作、任务采集、公开数据集

数据要覆盖多个任务类型：写作、摘要、问答、翻译、改写、代码等；要符合“人类风格”：客气、流畅、结构清晰

b)统一prompt

```
代码块
1  ### 指令：
2  你是一位写作助手，请帮我写一个关于环保的宣传标语。
3
4  ### 回答：
5  保护地球，从我做起，绿色生活每一天。
6
```

c) 微调方式选择：多数公司使用 LoRA

方法	说明	优缺点
全参数微调（Full Finetune）	修改所有模型权重	效果最强，但成本高、训练慢
LoRA	只训练插入的小模块	轻量、可部署、多任务共存

d) 开始训练：模型学会“做任务”

```
代码块
1  模型读取指令输入 → 学习如何生成正确回应
2  训练几轮（epoch），观察结果
```

(3) 指令微调后的模型能力变化：

能力项	预训练模型	微调后模型
语言能力	有	有
任务理解	弱	✅ 增强
风格控制	无	✅ 有风格、有语气
指令执行	弱	✅ 能写、能答、能翻译、能规划

3.5 微调超参数——算法去干

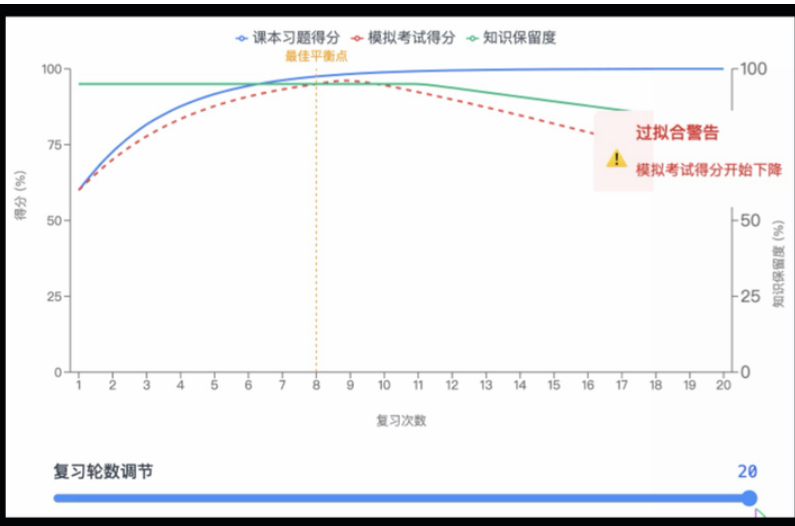
代码块

- 1 训练轮数（Epochs）、学习率（Learning Rate）、批量大小（Batch Size）
- 2 共同决定了模型 学习的速度、稳定性、效果和资源使用效率

(1) 训练轮数（Epochs）：一个 **Epoch** 表示模型完整看了一遍整个训练数据集
如果有10万条训练数据，设置 3个 Epoch → 模型将“遍历”这10万条数据 **3次**

代码块

- 1 太少会—学不够，精度低
- 2 太多会台死板，不知道变通
- 3 LoRA 一般1-3轮



超参数

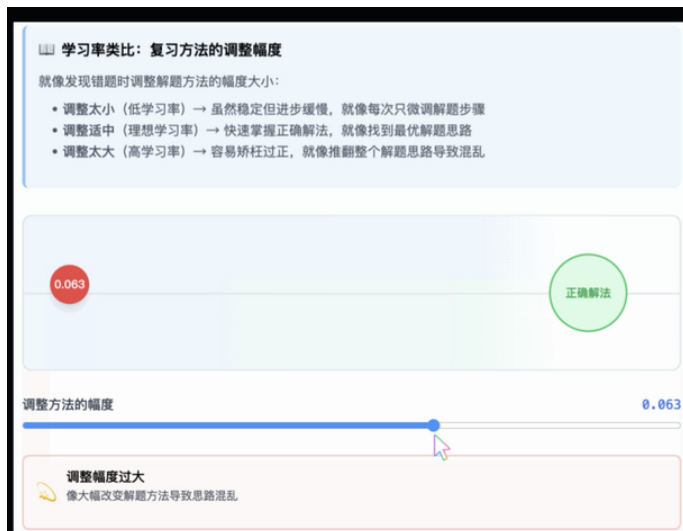
1、训练轮数（Number of Epochs）：
模型完整地遍历一次整个训练数据集的次数。
一个“Epoch”表示模型已经遍历了所有训练样本一次。
轮数少可能掌握的不太行（欠拟合）；
轮数多有可能“学傻了”不会变通（过拟合）

训练轮数就是我们从头到尾复习这本书的次数

(2) 学习率（Learning Rate）：控制模型在每一步参数更新时迈多大步

代码块

- 1 学习率 = 学习“急躁程度”
- 2 太大：走太快，容易错过最优点
- 3 太小：走太慢，训练很久还没学会



超参数

2、学习率（Learning Rate）：

决定模型在每次更新时参数调整的幅度，通常在 $(0, 1)$ 之间。也就是告诉模型在训练过程中“学习”的速度有多快。

学习率越大，模型每次调整的幅度越大；
学习率越小，调整的幅度越小。

学习率可以用来控制复习的“深度”，确保不会因为调整幅度过大而走偏，也不会因为调整幅度过小而进步太慢。如果你每次复习完一道题后，你会根据答案和解析调整自己的理解和方法

(3) 批量大小（Batch Size）：每次模型参数更新时，用多少条样本一块训练

代码块

- 1 举例：老师改作业，一次批改10份 vs 一次批改100份



超参数

3、批量大小（Batch Size）：

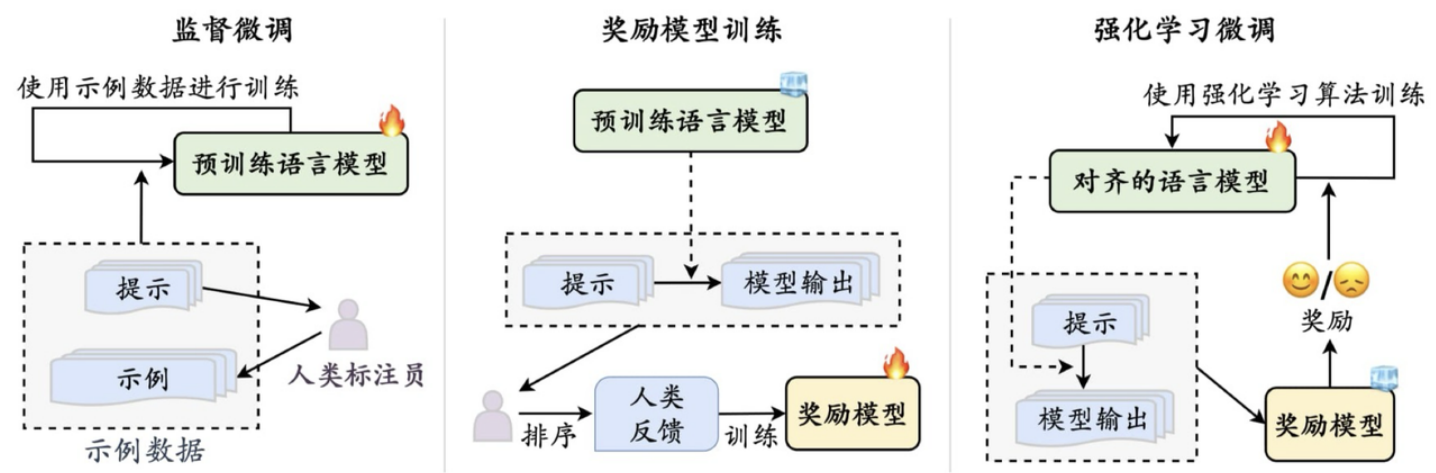
在模型训练过程中，每次更新模型参数时所使用的样本数量。它是训练数据被分割成的小块，模型每次处理一个小块的数据来更新参数

批量大小可以用来平衡复习速度和专注度，确保既能快速推进复习进度，又能专注细节。假设你决定每次复习时集中精力做一定数量的题目，而不是一次只做一道题

3.6 基于人类反馈的强化学习微调

RLHF:

- (1) 先用人类对模型输出的偏好反馈，来训练一个“奖励模型”，
- (2) 再用强化学习算法（如 PPO），引导大模型生成更符合人类偏好的答案。



1、2步走：

(1) 人类排序+训练奖励模型

给模型同一个提示，让它生成多个不同答案 → 让人类打分哪一个更好。

Prompt	回复1	回复2	人类偏好
“什么是碳中和？”	回答A：正确但机械	回答B：通俗易懂，有举例	✅ 选B

重复生成成千上万个这样的偏好数据。类似于训练一个 评分器，让它 “学会像人一样打分”

OpenAI 对reward model的标注界面：

Ranking outputs

To be ranked

B A team of researchers from Yale University and University of California, Davis studied the vocalization patterns of several different types of parrots. They found that parrots like to mimic human speech, and can produce a wide range of sounds, such as whistles, squawks, and other types of vocalizations...

Rank 1 (best)

A A research group in the United States has found that parrots can imitate human speech with ease, and some of them can even do so in the same way as humans. This group studied the sounds that parrots make in their natural habitats and found that they use their tongues and beaks in ways that are strikingly...

C Parrots have been found to have the ability to understand numbers. Researchers have found that parrots can understand numbers up to six. In a series of experiments, the parrots were able to identify the amount of food items under a number of cups...

Rank 2

Rank 3

E Scientists have found that green-winged parrots can tell the difference between two noises that are the same except for the order in which they are heard. This is important because green-winged parrots are known to imitate sounds. This research shows that they are able to understand the difference between sounds.

D Current research suggests that parrots see and hear things in a different way than humans do. While humans see a rainbow of colors, parrots only see shades of red and green. Parrots can also see ultraviolet light, which is invisible to humans. Many birds have this ability to see ultraviolet light, an ability

Rank 4

Rank 5 (worst)

(2) 通过强化学习优化主模型（比如 PPO 算法）

用 PPO 算法，引导原始大语言模型，在生成答案时追求“得高分”

代码块

- 1 PPO 做的事就是：
- 2
- 3 如果回答得分高 → 强化这种生成方式
- 4 如果回答得分低 → 惩罚并调整策略
- 5
- 6 → 大模型学会更稳妥、更委婉、更对齐人类偏好地回答问题

4、多模态大模型

4.1 开源 vs 闭源（同样适用于大语言模型 LLM）

代码块

- 1

开源大模型：指的是模型参数、结构、训练方式对外公开，允许他人下载、修改、部署甚至再训练
- 2

闭源大模型：指的是模型内部不可见，仅以API形式提供服务，用户不能访问模型权重或部署模型本体

对比维度	开源模型	闭源模型
模型参数	✅ 公开	❌ 不公开
模型结构	✅ 通常可查	✅ 部分保密
可部署性	本地/私有云都可	只能通过API访问
成本	一次下载/部署后无调用成本	按token/次数收费
可定制性	✅ 可微调、剪枝、合并	仅限API提供功能
数据安全	✅ 可私有化部署，不出网	所有输入上传云端
生态控制	社区主导（Meta、Mistral）	公司主导（OpenAI、Google）
示例模型	LLaMA、Qwen、Baichuan、Mistral	GPT-4、Claude、Gemini